



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 - Microprocessors

Interfacing using 8255 Programmable Peripheral Interface

Lecture 6-2

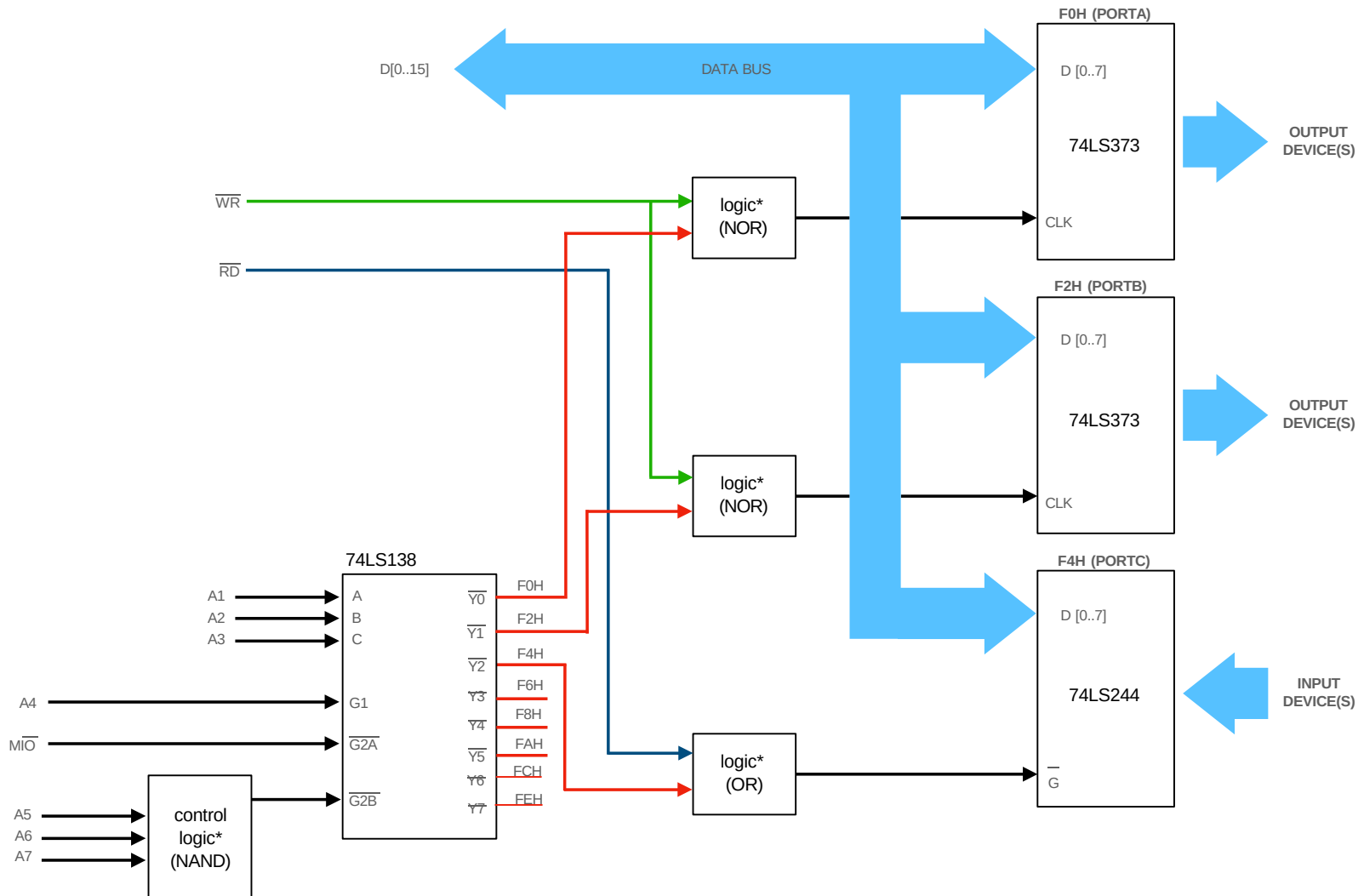
Unit 6: INPUT/OUTPUT (I/O) INTERFACING

© 2025

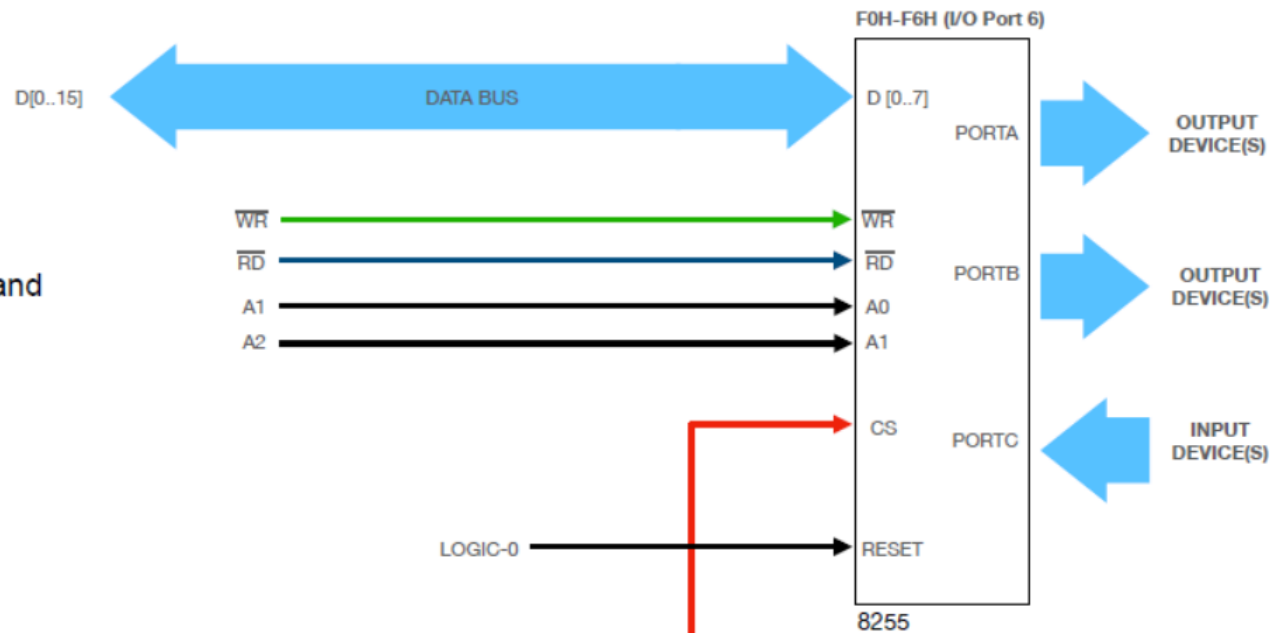
8255 Programmable Peripheral Interface

- It is an integrated circuit developed and manufactured by Intel in the first half of the 1970s for the Intel 8080 microprocessor.
- The 8255 provides 24 parallel input/output lines with a variety of programmable operating modes.
- The IC gives a CPU or digital system access to programmable parallel I/O.

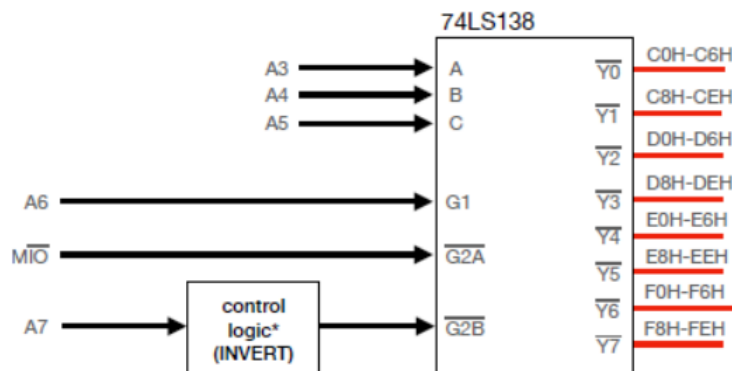
This configuration requires an IC for each I/O port excluding the control logic.

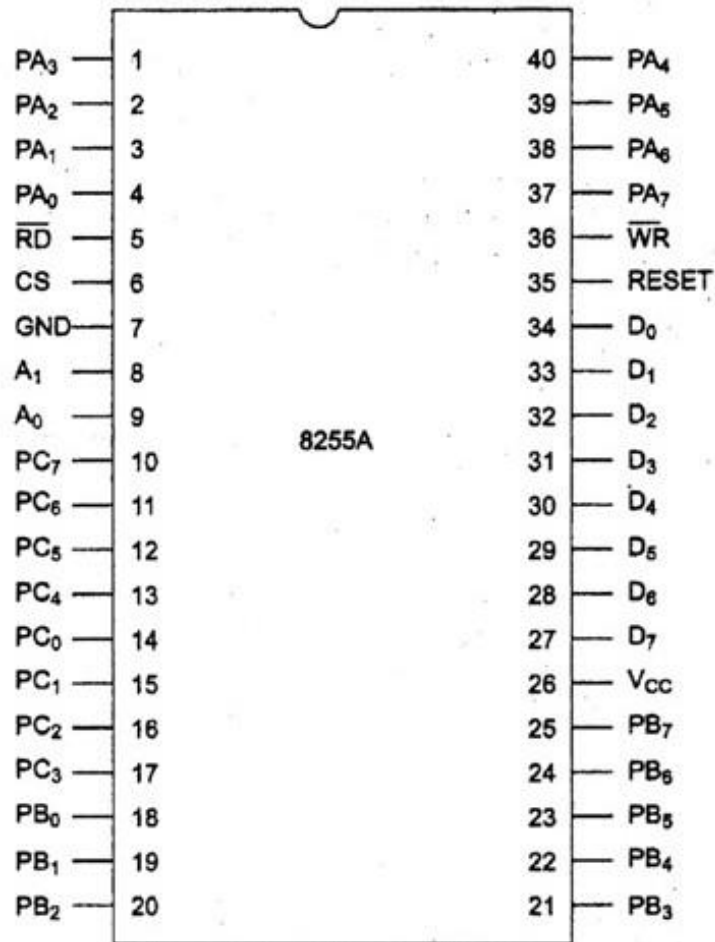


The 8255 replaces the latches and buffers.



Ports PORTA, PORTB & PORTC can be programmed either input or output. Each port is 8-bits wide.





D₇ - D₀ - Data Bus

PA₀ - PA₇ - Port A

PB₀ - PB₇ - Port B

PC₀ - PC₇ - Port C

A₀, A₁ - port select bits

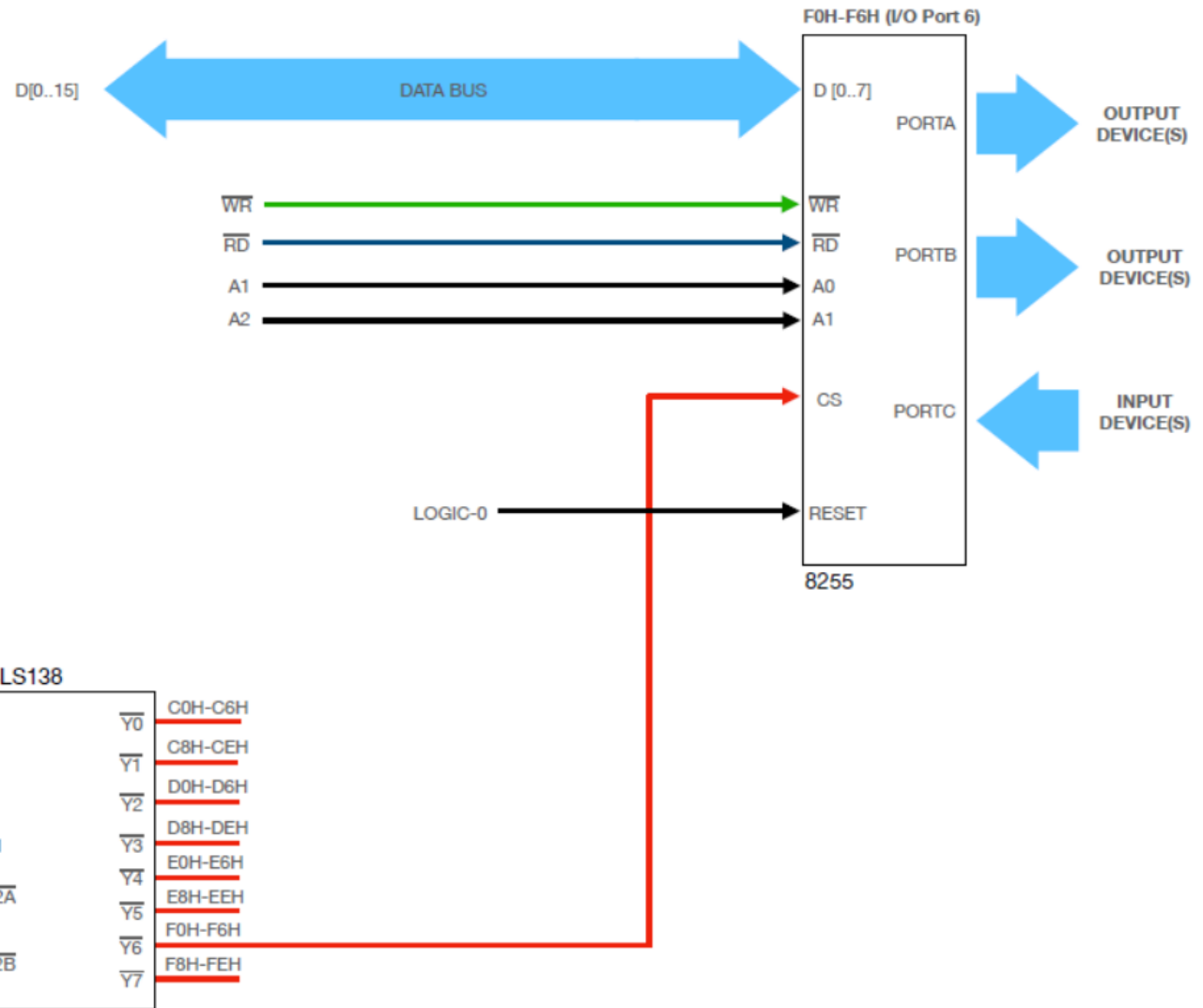
WR - logic-0 when write operation

RD - logic-0 when read operation

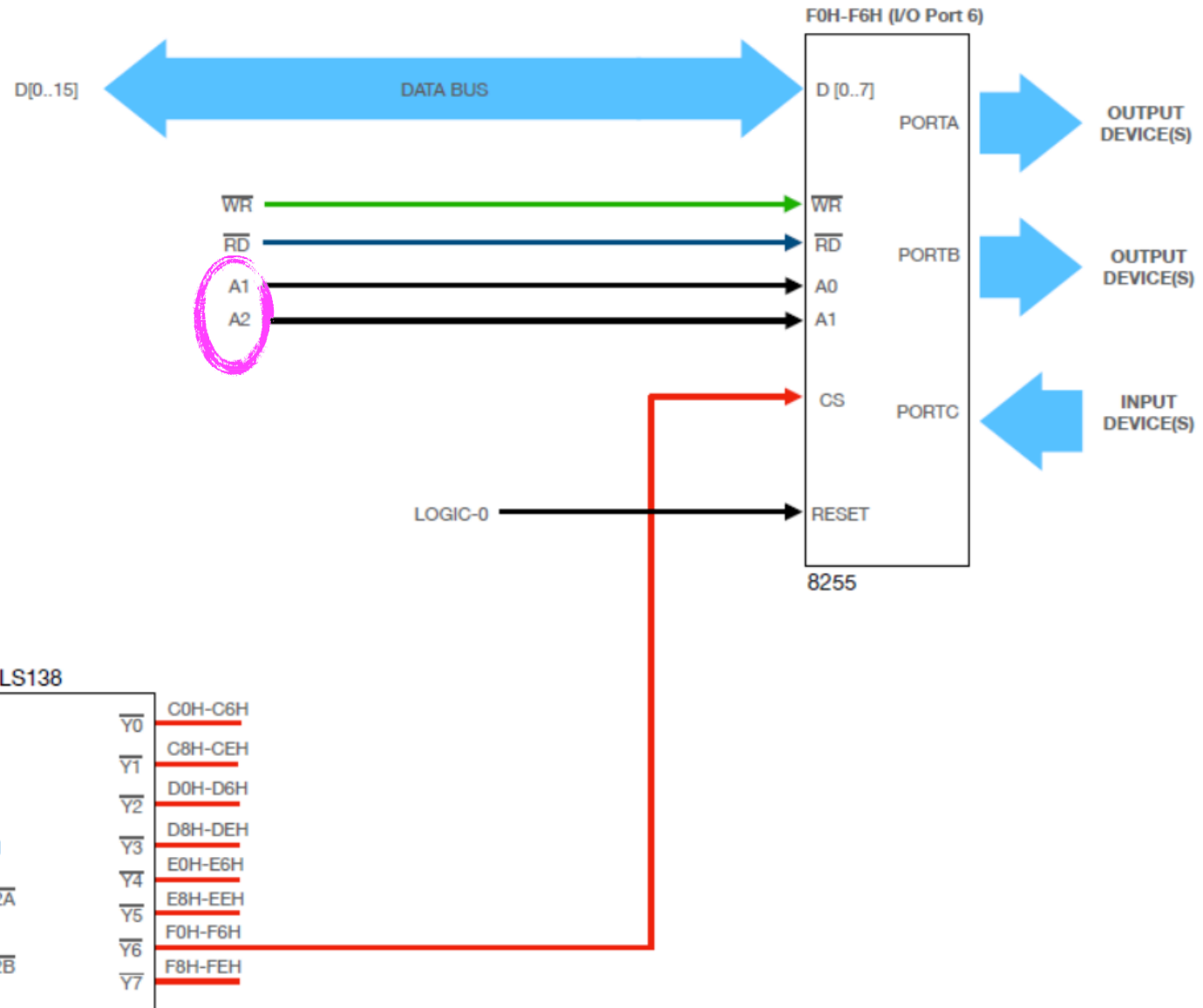
CS - chip enable

RESET* - resets the 8255 program

* The program of the 8255 will reset every time the power is turned off.



* RESET should be logic-0 for normal operation.



* RESET should be logic-0 for normal operation.

Programming the 8255

- Port A and port B can be used as 8-bit input/output ports.
- Port C can be used as an 8-bit input/output port or as two 4-bit input/output ports or to produce handshake signals for ports A and B.
- There are two basic operational modes of 8255:
 - a. Bit Set/Reset mode (BSR mode) and
 - b. Input/Output mode (I/O mode).

Port Selection

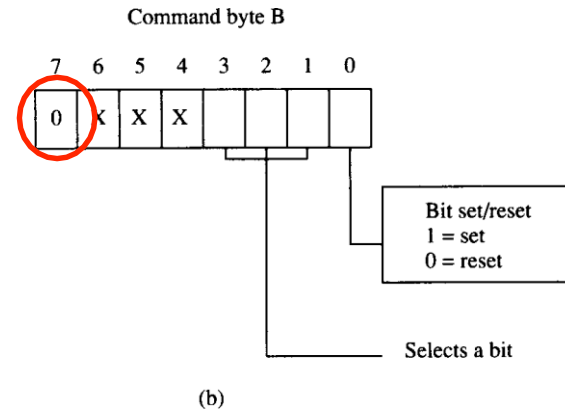
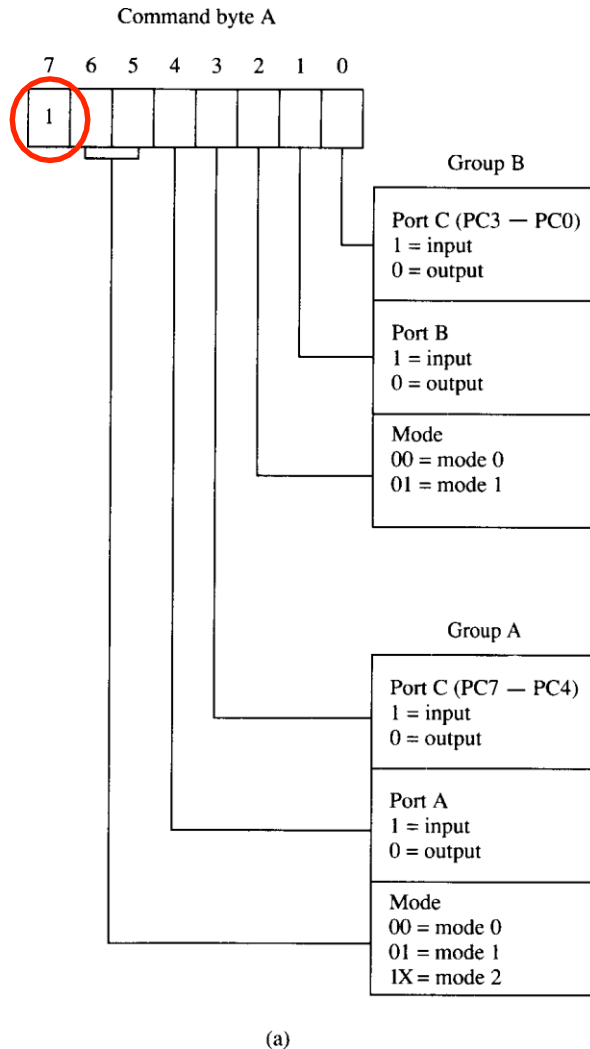
- Selecting the ports (PORTA, PORTB, PORTC) are controlled by the A_0 and A_1 pins of the 8255. The **Command Register*** can also be accessed.

A_1	A_0	<i>Function</i>
0	0	Port A
0	1	Port B
1	0	Port C
1	1	Command register

* This register is used to program the 8255 where the command byte is written

Command Byte

- The 82C55 is programmed through the two internal command registers.
- Bit #7 selects either command byte A or command byte B.
- **Command byte A** programs the function of group A and B, whereas **command byte B** sets (1) or resets (0) bits of port C only if the 82C55 is programmed in either mode 1 or 2.



Group B controls PB0 - PB7 and PC0 - PC3
Group A controls PA0 - PA7 and PC4 - PC7

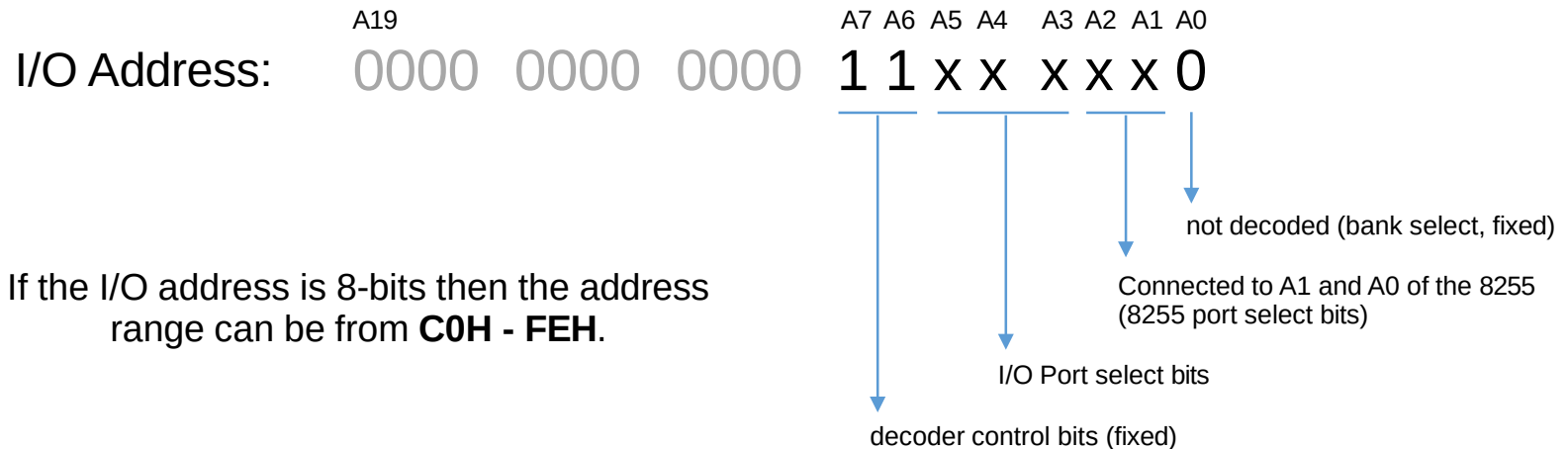
The command byte of the command register in the 82C55.
 (a) Programs ports A, B, and C. (b) Sets or resets the bit indicated in the select a bit field.

Mode 0 Operation

- This operation causes the 8255 to function either as a buffered input device or as a latched output device. These are the same as the basic input and output circuits discussed in the previous chapter.
- This also known as “I/O mode”.

Address Decoding

- Because part of the address line will be used to select the ports and command register, the address decoding will be handled differently.



Address Decoding

- Then, we need to determine the address for each I/O port and 8255 port select *combination*.

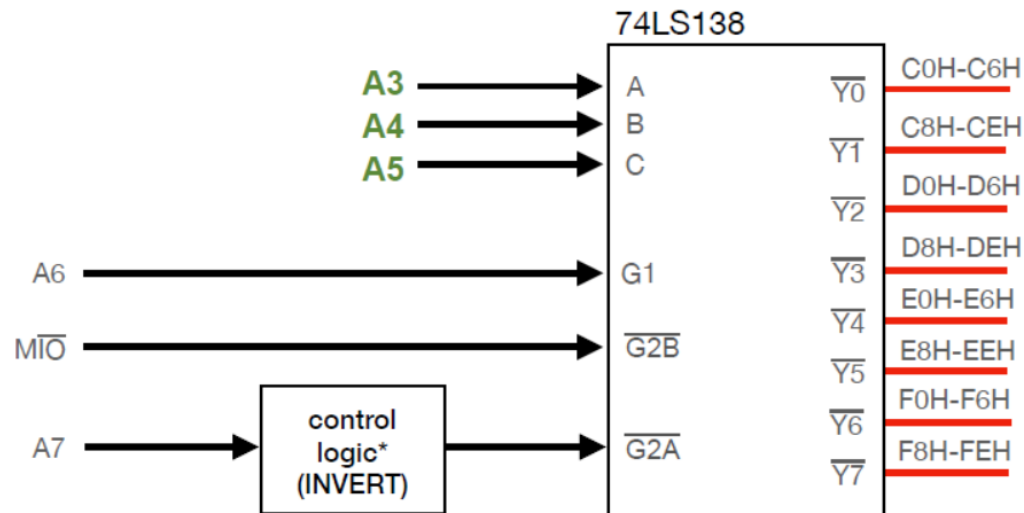
A7	A6	A5	A4	A3	A2	A1	A0	Address (Hex)	I/O Port (A5-A3)	8255 Port Selected (A2-A1)
1	1	0	0	0	0	0	0	C0H	PORT 0	PORTA
1	1	0	0	0	0	1	0	C2H		PORTB
1	1	0	0	0	1	0	0	C4H		PORTC
1	1	0	0	0	1	1	0	C6H		Command Register
1	1	0	0	1	0	0	0	C8H	PORT 1	PORTA
1	1	0	0	1	0	1	0	CAH		PORTB
1	1	0	0	1	1	0	0	CCH		PORTC
1	1	0	0	1	1	1	0	CEH		Command Register
1	1	0	1	0	0	0	0	D0H	PORT 2	PORTA
1	1	0	1	0	0	1	0	D2H		PORTB
1	1	0	1	0	1	0	0	D4H		PORTC
1	1	0	1	0	1	1	0	D6H		Command Register

* A7 and A6 are fixed at '1' and A0 is fixed at '0' to make sure the address is **even**.

A7	A6	A5	A4	A3	A2	A1	A0	Address (Hex)	I/O Port (A5-A3)	8255 Port Selected (A2-A1)
1	1	0	1	1	0	0	0	D8H	PORT 3	PORTA
1	1	0	1	1	0	1	0	DAH		PORTB
1	1	0	1	1	1	0	0	DCH		PORTC
1	1	0	1	1	1	1	0	DEH		Command Register
1	1	1	0	0	0	0	0	E0H	PORT 4	PORTA
1	1	1	0	0	0	1	0	E2H		PORTB
1	1	1	0	0	1	0	0	E4H		PORTC
1	1	1	0	0	1	1	0	E6H		Command Register
1	1	1	0	1	0	0	0	E8H	PORT 5	PORTA
1	1	1	0	1	0	1	0	EAH		PORTB
1	1	1	0	1	1	0	0	ECH		PORTC
1	1	1	0	1	1	1	0	EEH		Command Register
1	1	1	1	0	0	0	0	F0H	PORT 6	PORTA
1	1	1	1	0	0	1	0	F2H		PORTB
1	1	1	1	0	1	0	0	F4H		PORTC
1	1	1	1	0	1	1	0	F6H		Command Register
1	1	1	1	1	0	0	0	F8H	PORT 7	PORTA
1	1	1	1	1	0	1	0	FAH		PORTB
1	1	1	1	1	1	0	0	FCH		PORTC
1	1	1	1	1	1	1	0	FEH		Command Register

* A7 and A6 are fixed at '1' and A0 is fixed at '0' to make sure the address is **even**.

Address Decoding

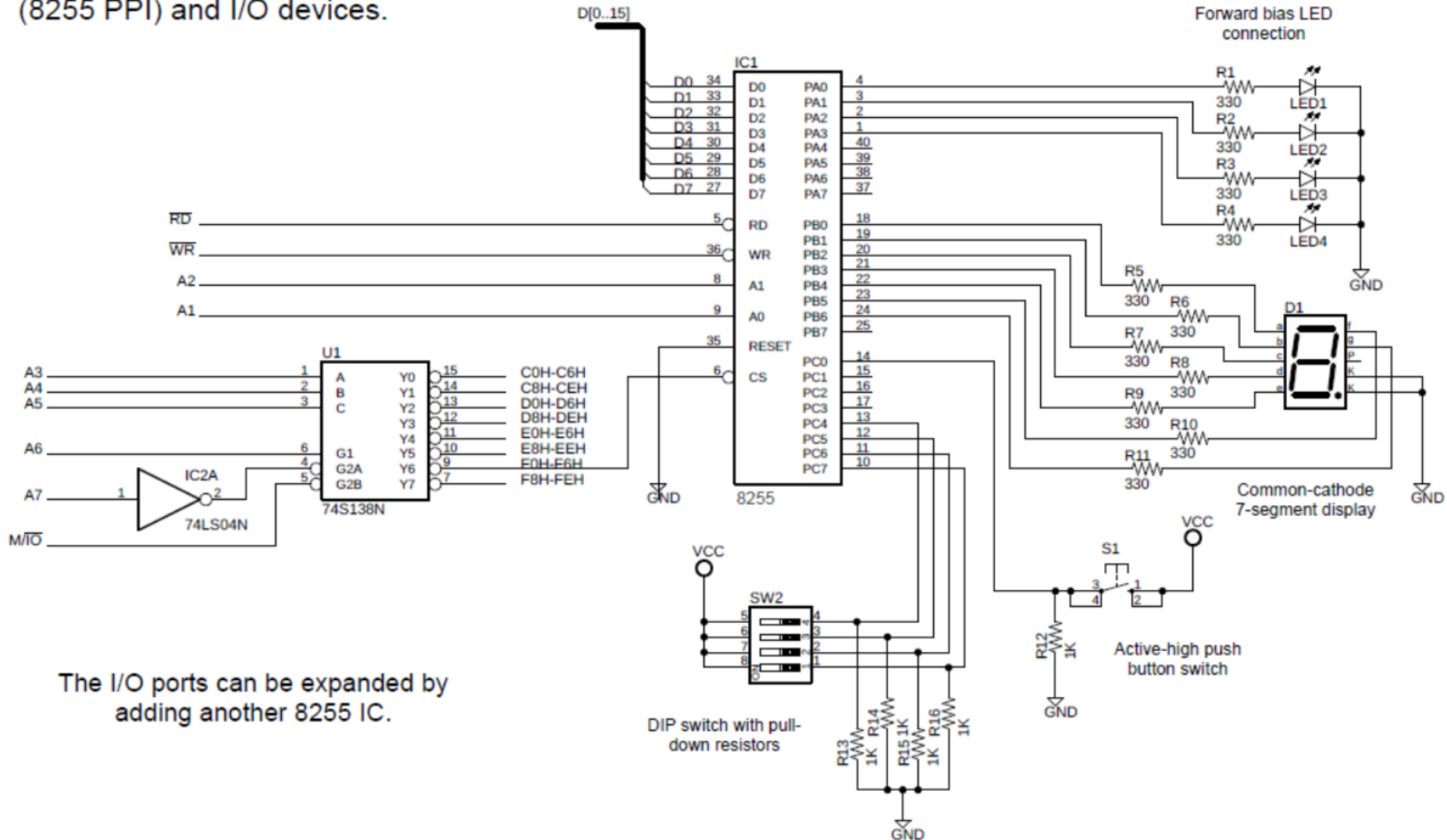


Based on the table, the address decoder could be designed as above.

To access PORTB of the 8255 at Port Address C0H-C6H (Port 0), I/O address should be: **C2H**.

To access the Command Register of the 8255 at Port Address C0H-C6H (Port 0), I/O address should be **C6H**.

Putting it all together: I/O
address decoder, I/O ports
(8255 PPI) and I/O devices.

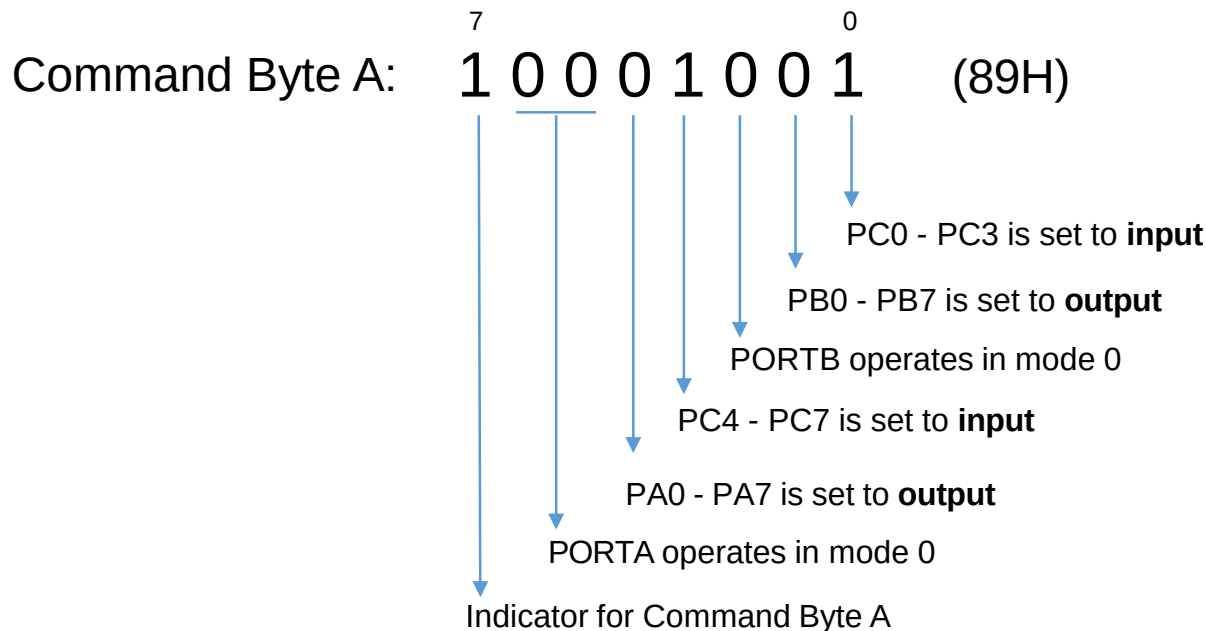


The I/O ports can be expanded by
adding another 8255 IC.

Command Byte

- Let's say that PORTA and PORTB are output ports and PORTC as input. All the ports operate in Mode 0:

In this example, we will only program the 8255 using the **Command Byte A** since PORTC operates in Mode 0.



Sending the Command Byte

- To send the command byte, set A1 = '1' and A0 = '1' then write the command byte to the address of the 8255.
- Let's send the command byte 89H to the 8255 at I/O Port 6 (F0H-F6H).

```
MOV DX, 0F6H ; sets the address to access command
              ; register of 8255 at Port 6 (see table)
MOV AL, 89H   ; move command byte to AL
OUT DX, AL    ; send command byte
```

- The 8255 must be programmed first before using, accessing its I/O ports.

Writing/Reading Data

- To write data, set the address to select the correct port in 8255 then send the data.

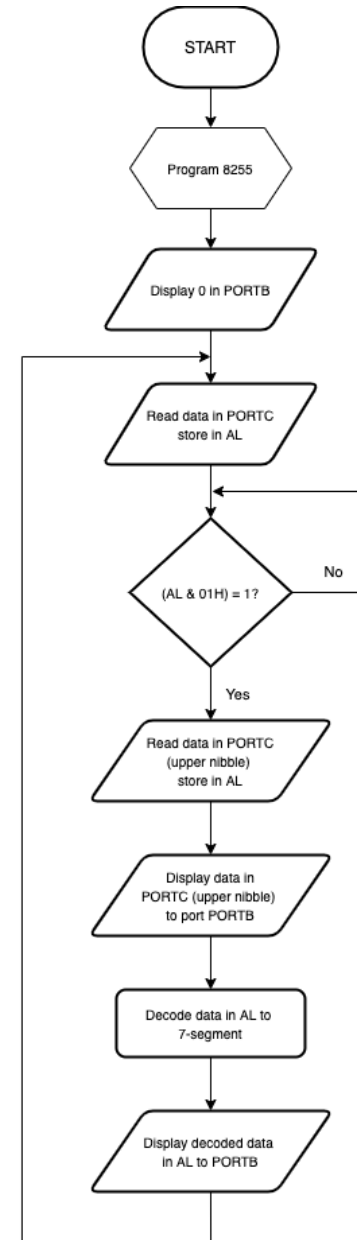
```
MOV DX, 0F0H ; sets the address to access PORTA of 8255
               ; at Port 6 (see table)
MOV AL, 0FH  ; set data to be sent to PORTA
OUT DX, AL   ; send data to PORTA
```

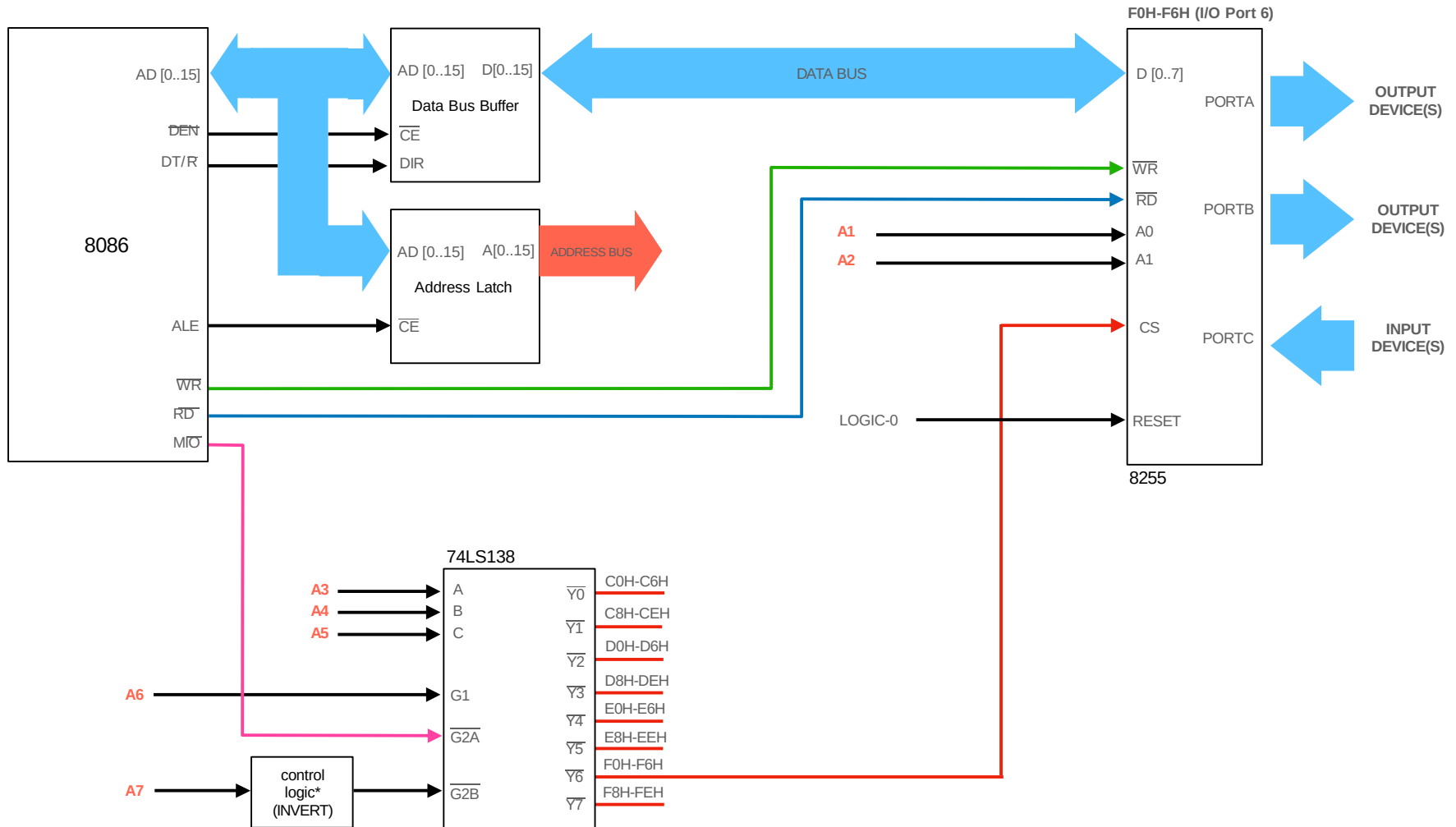
- To read data, set the address to select the correct port in 8255 then read the data.

```
MOV DX, 0F4H ; sets the address to access PORTC of 8255
               ; at Port 6 (see table)
IN AL, DX    ; read data from PORTC
```

Practice Exercise

- Circuit on page 17 will be interfaced to the 8086 in Proteus.
- See flowchart to the right for the program.





Interfacing 8255 to 8086 in Proteus

```

DATA SEGMENT
    PORTA EQU 0F0H          ; address to access PORTA
    PORTB EQU 0F2H          ; address to access PORTB
    PORTC EQU 0F4H          ; address to access PORTC
    PORT_CON EQU 0F6H       ; address to access Command Register
DATA ENDS

CODE SEGMENT PUBLIC 'CODE'
    ASSUME CS:CODE
START:
    ; Program 8255 (PORTA & PORTB as output, PORTC as input)
    MOV DX, PORT_CON        ; set port to Command Register
    MOV AL, 89H             ; set command byte
    OUT DX, AL              ; send data in AL to Command Register
    ; Display '0' in PORTB
    MOV DX, PORTB           ; set port to PORTB
    MOV AL, 00111111B       ; set 7-segment data to display '0'
    OUT DX, AL              ; send data in AL to PORTB
THERE:
    ; check push-button switch
    MOV DX, PORTC           ; set port to PORTC
    IN AL, DX               ; read data in PORTC
    TEST AL, 01H            ; check bit 0 if logic 1 (switch)
    JZ THERE                ; jump to THERE if bit 0 is not equal to 1
    ; read data from DIP switch
    MOV DX, PORTC           ; set port to PORTC
    IN AL, DX               ; read data in PORTC
    MOV CL, 4
    SHR AL, CL              ; move the data to the lower nibble
    ; display data from DIP switch to LEDs
    MOV DX, PORTA           ; set port to PORTA
    OUT DX, AL              ; send data in AL to PORTA
    ...

```

```

; Decode data in PORTC (data in AL)
MOV AH, 00H          ; set AH to 00H (upper byte of AX must be 00H)
MOV SI, AX           ; set SI (offset) to value of AX
MOV AL, S_SEG[SI]    ; get data from SEG with offset SI
; Display to 7-segment display in PORTB
MOV DX, PORTB        ; set port to PORTB
OUT DX, AL           ; send data in AL to PORTB
JMP THERE            ; check push-button switch again and repeat process

; 7-segment data
S_SEG DB 00111111B    ; 0
      DB 00000110B    ; 1
      DB 01011011B    ; 2
      DB 01001111B    ; 3
      DB 01100110B    ; 4
      DB 01101101B    ; 5
      DB 01111101B    ; 6
      DB 00000111B    ; 7
      DB 01111111B    ; 8
      DB 01100111B    ; 9
CODE ENDS
END

```



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 - Microprocessors

End of Lecture

Unit 6: INPUT/OUTPUT (I/O) INTERFACING

© 2025